Ders 25 Genişletilmiş Notu: Alt Dizgi Arama: Kaba Kuvvet, Horspool, Boyer-Moore ve Rabin-Karp

Ders 25 Genişletilmiş Notu: Alt Dizgi Arama: Kaba Kuvvet, Horspool, Boyer-Moore ve Rabin-Karp

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Biçimi
- 2. Alt Dizgi Arama Problemi
- 3. Alt Dizgi Arama Nerelerde Kullanılır?
- 4. Kaba Kuvvet (Brute Force) Arama
- 5. Horspool Algoritması
- 6. Boyer-Moore Algoritması
- 7. Rabin-Karp Algoritması
- 8. Algoritmaların Karşılaştırılması ve Seçim
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Biçimi

Hoca bu dersi canlı anlatmadı. Önceki yıllardan birinin ders kaydını bu dersin konusuna göre kesip düzenlemiş, derste ekran paylaşımıyla izletti. Hocanın sözüyle, konu eksik kalmayacak; öğrenciler bir önceki yılın dersini dinlemiş gibi olacak. Bu yüzden notun 2. bölümden itibaren anlatımı, izletilen kayda aittir. Kayıtta hoca slaytların üzerine kalemle çizerek anlatıyor.

Dersin konusu karakter dizileri (string) üzerinde arama, yani alt dizgi arama (substring search) yöntemleridir. Hoca bütün yöntemleri incelemeyeceklerini, çok iyi çalışan ve yaygın kullanılan "üç buçuk" algoritmaya bakacaklarını söyledi. Buçuk olan, Horspool'dur; çünkü Horspool, Boyer-Moore'un sadeleştirilmiş hâlidir.

2. Alt Dizgi Arama Problemi

Problem şudur: Bize n uzunluğunda bir metin (text) veriliyor. Bu metnin içinde m uzunluğunda bir metni bulmak istiyoruz. Aranan bu kısa metne genellikle desen (pattern) ya da örüntü denir. Metin tek boyutlu olduğu için "boyut" yerine "uzunluk" demek daha doğrudur.

Genellikle problemlerde n, m'den çok büyüktür ($n \gg m$). Yani elimizde çok uzun bir veri vardır ve bu verinin içinde nispeten kısa, bazen bir kelime uzunluğunda örüntüleri ararız.

Metnin sabit uzunlukta olması da gerekmez. Sonsuz uzunlukta akan veri (stream) üzerinde de arama yapabilmeliyiz. Hocanın örneği Twitter'dı: mesajlar sürekli akıyor. Bir başka benzetme olarak, ağ üzerinden sürekli yazı yollandığını, telgraf mesajı gibi harflerin "tık tık" peş peşe gelip bir katar oluşturduğunu düşünebiliriz. Bu akan katarın içinde "needle" (iğne) kelimesini aramak, samanlıkta iğne aramaya benzer: "… needle haystack needle haystack …".

Veri büyüdükçe, yani n büyüdükçe, örüntüyü hızlı ve verimli bulmak önem kazanır. Örüntünün hangi adreste (indiste) geçtiğini, bir sonraki geçişin nerede olduğunu ya da kaç kez geçtiğini

bulmak isteyebiliriz. Hocanın örneği: Milyonlarca satırlık bir kodda `public` anahtar kelimesinin kaç kez geçtiğini saymak istiyorsunuz. Bunu nasıl, hangi gezintiyle yapacağımız sorusu alt dizgi arama problemidir.

3. Alt Dizgi Arama Nerelerde Kullanılır?

Hoca problemi kullanım alanlarıyla somutlaştırdı:

- **Web aramaları:** Arama doğal olarak bir sayfanın içinde kelimenin geçip geçmediğine bakar.
- **Veri tabanı sorguları:** Adı, soyadı belirli olanları ya da doğum yılı belirli olanları seçmek. Hoca kendi adını örnek verdi: adı "Göksel" olanları arıyorsak, sorgu motoru bütün ad kolonlarını bu kelimeyle karşılaştırmak zorundadır.
- **İntihal (plagiarism) tespiti:** Birebir aynı örüntünün geçtiği parçalar olabilir. Ama biri bir cümleyi alıp araya kelimeler eklemiş olabilir. Bu durumda desen küçük parçalara bölünür ve parçalar aranır. Böylece benzer ya da yakın uzaklıktaki örüntüler de yakalanır.
- **Adli bilişim (forensics):** Elinizde bir disk ya da bellek görüntüsü (imajı) vardır. İçinde belirli imzaları ararsınız. Örneğin belirli bir RSA anahtarı geçiyor mu? Ya da diskte ne kadar web sitesi giriş kaydı, ne kadar URL var? Bunun için `http` ya da `https` ile başlayan dizgileri terabaytlarca diskte ararsınız. Bu ölçekte hızlı ve etkili bir arama algoritması şarttır.
- **Spam tespiti:** Spam belirtisi olan örüntüler aranır ("profits", "lose weight", "Viagra", "There is no catch. This is a one time mailing.", "This message is sent in compliance with spam regulations" gibi). Spam gönderen, arama dizgilerine yakalanmamak için O yerine sıfır, I yerine bir yazabilir; font da bunu gizleyebilir. Tam eşleşme (exact match) arayan bir sistem bunu kaçırır. Bu yüzden bu varyantlar da örüntü olarak üretilip e-posta gövdelerinde aranır. Görüldüğü gibi örüntü tek kelime de olabilir, bir cümle uzunluğunda da. Algoritmanın hepsinde iyi çalışması beklenir.
- **Trafik izleme:** Hocanın hikâyesinde bir istihbarat ajanı, güvenlik için bütün internet trafiğini izlemek ister. Yasa koyucu, iletişimin gizliliği nedeniyle buna izin vermez. Ajan, aslında yalnız planlanan bir saldırıyla ilgili konuşmayı bulmak istediğini, bunun için "attack at dawn" (şafakta saldır) desenini aradığını söyler. Yasa koyucu da bütün trafiği izlemesine değil, yalnız bu deseni tarayan ve bulduğu yeri işaretleyen bir makine yapmasına izin verir. Makine desenin geçtiği yeri bulunca, mesajın nereden nereye gittiği ayrıca takip edilir.
- **Bilgi çıkarımı (information extraction):** Hocanın örneği bir Yahoo finans sayfasıydı. Google hissesinin son işlem değerini ("Last Trade") okumak istiyoruz. HTML kodunda önce "Last Trade" ifadesi aranır ve konumu bulunur. Sonra bu konumdan sonraki ilk kalın yazı etiketi `<b>` ve kapanış etiketi `</b>` aranır. Başka `<b>` etiketleri de olabilir, ama "Last Trade"den hemen sonrakini aldığımız için doğru yeri buluruz. İki etiketin arasındaki alt dizgi okunur ya da kopyalanır; bu, arama anındaki hisse değeridir. Hoca, örneği basit tutmak için sayfa kodunu sadeleştirdiğini, bu yüzden ekrandaki sayfa ile koddaki değerin farklı olduğunu belirtti.

4. Kaba Kuvvet (Brute Force) Arama

4.1 Algoritma

İlk akla gelen, her zaman olduğu gibi, hiçbir ön işlem yapmadan "dümdüz" aramaktır. Desen metnin başına yerleştirilir ve karakterler karşılaştırılır. Birinci karakter tutarsa ikinciye, sonra m'inci karaktere kadar teker teker geçilir. m karakterin hepsi eşleşirse desen bulunmuştur; konumu kaydırma (shift) indisinden okunur.

Eşleşme olmazsa, ilk karakter tutmamış da olsa, birkaç karakter tutup sonra bozulmuş da olsa, desen yalnızca **bir** karakter kaydırılır. Daha fazla kaydırırsak bir eşleşmeyi kaçırabiliriz. Metnin sonuna kadar böyle gidilir; bulunamazsa "bulunamadı" denir. Hocaya göre Java'daki `String.indexOf` de bu şekilde, teker teker karşılaştırarak arar.

Slayttaki sözde kod CLRS'teki biçimdedir: kaydırma s, 0'dan $n-m$'ye kadar gider. Son değer $n-m$'dir, çünkü desenin son karakteri metnin son karakterine denk gelene kadar denemek gerekir. $P[1..m] = T[s+1..s+m]$ ise "desen s kaydırmasında geçiyor" yazdırılır. Hoca, döngü

bittikten sonra bir satır ekleyip `return -1` ya da "not found" yazdırılabileceğini söyledi. Bütün alt dizgi arama algoritmalarında karmaşıklık n ve m üzerinden hesaplanır.

Aynı fikir, iç döngüde j ile desen üzerinde ilerleyerek C'de şöyle yazılabilir:

```c
#include <string.h>

/* Desenin metindeki ilk geçtiği indisi döndürür, yoksa -1. */
int bruteForceSearch(const char *text, const char *pat) {
    int n = strlen(text), m = strlen(pat);
    for (int i = 0; i <= n - m; i++) {
        int j = 0;
        while (j < m && text[i + j] == pat[j])
            j++;
        if (j == m)
            return i;          /* eşleşme i indisinde */
    }
    return -1;                 /* bulunamadı */
}
```

Hoca aynı akışı bir akış diyagramıyla (flowchart) da gösterdi.

4.2 Örnek: ACAABC içinde AAB

Metin `ACAABC`, desen `AAB`:

- $s = 0$: `A` = `A` tuttu. İkinci karakter: metinde `C`, desende `A`; tutmadı. Bir kaydır.
- $s = 1$: `C` ile `A` tutmadı. Bir kaydır.
- $s = 2$: `A` = `A`, `A` = `A`, `B` = `B`. Üçü de tuttu; desen 2 indisinde bulundu.

İstenirse metnin sonuna kadar devam edilir, çünkü başka geçişler de olabilir. Bu kısa örnekte bir deneme daha kalır: $s = 3$'te `A` = `A` tutar, sonra metindeki `B` ile desendeki `A` tutmaz ve metnin sonuna gelinir. Geçişleri saymak istersek bir sayaç tutabiliriz.

4.3 Örnek: SIMPLE araması ve neden yalnız bir kaydırılır?

Hoca karşılaştırmaları tek tek söyleyerek ikinci bir örnek yaptı. Desen `SIMPLE` (6 karakter).

Deneme	Karşılaştırma	Sonuç
1	`T` - `S`	Tutmadı, 1 kaydır
2	`H` - `S`	Tutmadı, 1 kaydır
3	`I` - `S`	Tutmadı, 1 kaydır
4	`S` - `S` tuttu; boşluk - `I`	Tutmadı, 1 kaydır
5	boşluk - `S`	Tutmadı, 1 kaydır
6	`I` - `S`	Tutmadı, 1 kaydır
7	`S` - `S` tuttu; boşluk - `I`	Tutmadı, 1 kaydır
8	boşluk - `S`	Tutmadı, 1 kaydır
9	`A` - `S`	Tutmadı, 1 kaydır
10	boşluk - `S`	Tutmadı, 1 kaydır
11	`S`, `I`, `M`, `P`, `L`, `E`	6 karakter de tuttu, bulundu

"THIS IS A SIMPLE …" metninde bu, 10 indisine karşılık gelir. Diğer geçişleri bulmak için yine birer kaydırarak metnin sonuna kadar devam edilir.

Soru: Madem tutmadı, deseni daha çok kaydıramaz mıyız? **Cevap:** Kaydıramayız. Çünkü "SIMPLE" bir sonraki karakterden ya da ondan sonrakinden itibaren başlıyor olabilir; fazla kaydırırsak onu kaçırırız. Kaba kuvvette mutlaka birer birer kontrol yapmak zorundayız.

4.4 En kötü durum

En kötü durum, verinin ve örüntünün tekrarlı olduğu durumdur. Desen `AAAAB`, metin `AAAA...AAB` olsun; aranan şey en sonda çıkar. Her konumda ilk dört karşılaştırma tutar, beşinci (`B` ile `A`) tutmaz. Yani her denemede 5 karşılaştırma yapılır, sonra yalnız bir kaydırılır. Yaklaşık n (tam olarak $n - m + 1$) konum denenir ve her birinde m karakter karşılaştırılır. Toplam yaklaşık $m \cdot n$ karşılaştırma yapılır.

m büyüdükçe durum kötüleşir. Örneğin m, n'ye yakınsa karmaşıklık tavanı n^2'ye doğru gider. Ama m genellikle n'den çok küçük olduğu için bunu ayrıca mn olarak göstermeyi tercih ediyoruz. Büyük O gösterimi en kötü durumu esas aldığı için kaba kuvvet aramanın karmaşıklığı $O(mn)$'dir.

4.5 Akan veri ve geri dönüş alanı (backup)

Statik veride kaba kuvvetle ileriye doğru arama yapılabilir. Ama akan veride "tuttu, tuttu, tuttu, son karakter tutmadı; geri dön ve bir kaydır" diyemeyiz. Biz geri dönüp bir kaydırana kadar karakterler akmaya devam eder. Örneğin desenin ilk birkaç karakteri tuttuktan sonra son karakter tutmazsa, o ana kadar okunan karakterleri bir ön bellekte tutmamız gerekir ki yeni gelen karakterleri kaçırmayalım. En fazla $m - 1$, son karşılaştırmayla birlikte m karakterlik bu alana geri dönüş alanı (backup) denir.

Hoca istihbarat ajanı ve yargıç hikâyesine döndü: Akan veride bu kadar bilgiyi saklayacak yerimiz olmayabilir. Verinin hızı yüzünden zamanımız olmayabilir. Ya da yasa "akan veriden aradığın kısım kadar bir parçayı bile tutamazsın" diyebilir; iznimiz olmayabilir. O zaman başka bir arama yöntemi gerekir.

Geri dönüş alanı tutabiliyorsak, kaba kuvvet tek döngüyle şöyle yazılır. i ve j sıfırdan başlar. Döngü $i < n$ ve $j < m$ olduğu sürece devam eder. Metin karakteri desen karakteriyle eşleşirse j artırılır; böylece desenin ikinci, üçüncü, ..., m'inci karakterine geçilir. Eşleşme yoksa `i -= j` yapılır ve j sıfırlanır. `i -= j`, okunan j karakter kadar geri gelmek, yani o kadar alanı "backup"lamaktır. Döngü bitince $j = m$ ise desen $i - m$ indisindedir. j, m'ye ulaşmadıysa döngüden metnin bitmesi yüzünden çıkılmıştır ve desen metinde yoktur.

```c
#include <string.h>

/* Geri dönüş alanlı (backup) kaba kuvvet arama. */
int bruteForceBackup(const char *text, const char *pat) {
    int n = strlen(text), m = strlen(pat);
    int i, j;
    for (i = 0, j = 0; i < n && j < m; i++) {
        if (text[i] == pat[j])
            j++;
        else {
            i -= j;            /* j karakter geri dön (backup) */
            j = 0;
        }
    }
    if (j == m)
        return i - m;          /* bulundu */
    return -1;                 /* bulunamadı */
}
```

Eşleşme yokken `i -= j` yapıldıktan sonra döngünün `i++` adımı çalışır. Böylece bir sonraki deneme, önceki denemenin bir sağından başlar (ara adım eklendi).

Geri dönüş alanı kullanmayan yöntemlerin en bilinenlerinden biri KMP (Knuth-Morris-Pratt) algoritmasıdır. Hoca KMP'yi bu ders kapsamında anlatmayacağını söyledi. Onun yerine daha yaygın kullanılan ve birçok arama çubuğunun arkasında farkında olmadan kullandığımız Horspool ve Boyer-Moore'a geçti.

5. Horspool Algoritması

5.1 Fikir

Kaba kuvvet sağlam bir yöntemdir ama karmaşıklığı mn'ye kadar çıkar. Zayıf karnı, eşleşme bozulduğu anda ne olursa olsun yalnız bir ilerlemesidir. Hepimizin aklına şu soru gelir: "Nasıl olsa tutmadı; deseni daha fazla kaydıramaz mıyım?" Hoca bir fikir daha ekledi: Desene baştan değil sondan, yani sağdan sola baksak ve eşleşme olmadığında daha fazla karakter kaydırabilsek? Horspool algoritmasının makalesi bu fikirle 1980'de yayımlanmıştır.

Slaytta başlık olarak Horspool yazıyor; Boyer ve Moore adları açık gri yazılmış. Hocaya göre gerçek algoritma Boyer-Moore'dur, Horspool onun basit, hafifletilmiş versiyonudur. Hoca bir ipucu da verdi: İnternette bu algoritmaların kodlarına bakarken kafanız karışabilir. Bu konuda tam bir uzlaşma (konsensüs) yoktur; Horspool diye Boyer-Moore'u yazan çok kaynak vardır. Aradaki farkı iyi bilirseniz kaynakları ve kodları daha rahat süzersiniz.

Horspool, kötü karakter sezgisi (bad character heuristic), yani eşleşmeyen karakter sezgisi olarak da bilinir. Amaç, eşleşme olmadığında bir yerine üç, beş, mümkünse desen uzunluğu kadar kaydırıp olabilecek en yüksek hıza çıkmaktır.

Örnek: Desen `ABC` olsun. Önce desenin son karakteri `C`, metindeki hizadaki karakterle karşılaştırılır. Metinde `E` çıktı. `E` desende hiç yoktur. O zaman garantili olarak deseni o karakterin sonrasına kadar "lap diye" taşıyabiliriz; yani m karakter birden geçebiliriz. Karakter desende varsa m'den az, ama 1'den fazla (en kötü durumda 1) bir x kadar kaydırırız.

Sorun, hiçbir eşleşmeyi kaçırmamayı garanti eden en büyük kaydırma miktarını nasıl bulacağımızdır. Yolu bir kötü eşleşme tablosu (bad match table) oluşturmak ve metinde görülen karakterin tablodaki değeri kadar kaydırmaktır. Bu sayede genellikle doğrusal zamanda kalırız; en kötü durumda yine mn'ye çıkabiliriz.

5.2 Dört durum

Tabloyu kurmak için önce olası durumlara bakmak gerekir. c, desenin son karakterinin hizasındaki metin karakteridir.

Durum 1: c desende yok. Desen `BARBER`. Sondan başlayarak karşılaştırıyoruz ve metinde `S` görüyoruz. `S` desende olmadığı için, bu 6 karakterin hiçbir hizasında eşleşme yakalama olasılığımız yoktur. Deseni boyu kadar, yani 6 karakter birden kaydırırız. Kaba kuvvet burada 1 kaydırırdı. Olabilecek en iyi kaydırma budur.

Soru: Sondan karşılaştırıyorsak ve veri akan veriyse, desenin önceki kısmına denk gelen karakterler daha önce gelmiş olmaz mı? **Cevap:** Evet. Son gelen veri, desenin son karakterinin hizasındakidir. Önceki karakterlerle de karşılaştırma yapıldığı için Horspool da m (en az $m-1$) karakterlik küçük bir ön bellek, yani geri dönüş alanı ister.

Durum 2: c desende var ama son karakterle eşleşmedi. Desen `BARBER`, son karakter `R`; metinde `B` gelmiş. Eşleşme yok ama `B` desende var. Desende iki `B` vardır, iki seçenek doğar:

- En sağdaki `B`'yi metindeki `B`'nin altına getirmek. Bu 2 karakter kaydırmak demektir. Bir karakterden yine iyidir.
- Soldaki `B`'yi oraya getirmek. Bu daha çok kaydırır ama tehlikelidir. Metinde tam o noktada `BARBER` başlıyor olabilir; o zaman onu kaçırırız.

Bu yüzden karakter, desendeki **en sağdaki** tekrarıyla hizalanacak kadar kaydırılır. Bunun en kötü durumu 1 kaydırmaktır. Örneğin metindeki karakter `E` olsaydı, `E`'yi hizalamak için yalnız 1 karakter kaydırırdık; kaba kuvvet gibi olur.

Durum 3: c desenin son karakteriyle eşleşti ve geri kalan $m-1$ karakterde başka tekrarı yok. Desen `LEADER`. `R` tuttu, `E` tuttu, sonra metindeki `M` ile desendeki `D` tutmadı. Desen yine boyu kadar, 6 karakter kaydırılır. Üç karşılaştırma yapmış olsak da 6 karakter ileri atlarız.

Durum 4: c son karakterle eşleşti ve geri kalan kısımda tekrarı var. Desen `REORDER`, metinde `...AR` gelmiş. `R` tuttu, sonra `A` ile `E` tutmadı. "`A` bende yok, direkt kaydırayım" diyemeyiz, çünkü kaydırmayı belirleyen hizadaki `R`'dir ve `R` desende başka yerlerde de geçer. Durum 2'de olduğu gibi, desenin geri kalanındaki en sağdaki `R` bu `R`'nin altına oturtulur.

Burada “A desende yok, daha fazla kaydırabilirdik” itirazı haklıdır. Normalde daha ileriye kaydırılabilirdi. Ama Horspool tabloyu tek bir kuralla kurduğu için bu durumda olması gerekenden az kaydırır. Hocaya göre bu, Horspool’un zayıf ya da “kaybettiren” noktasıdır. Yine de kaba kuvvetten çok daha iyidir: en kötü durumda 1, genellikle çok daha fazla kaydırır.

> **Ders sonrası not:** Hoca REORDER örneğinde desenin 4 karakter kaydırıldığını söyledi. Son R hariç en sağdaki R, $j = 3$’tedir; tablo değeri $7 - 1 - 3 = 3$’tür. Doğrusu 3 karakter kaydırmaktır (python ile doğrulandı).

> **Ders sonrası not:** LEADER örneğinde hoca kaydırmayı “M desende yok” diye açıkladı. Horspool’da kaydırma her zaman desenin son karakteri hizasındaki metin karakterine göre bulunur; burada o karakter R’dir. R’nin desende başka tekrarı olmadığı için tablo değeri 6’dır ve sonuç aynıdır.

5.3 Kaydırma tablosu (shift table)

Her karşılaştırmada bu durumları tek tek hesaplamak yerine bir karakter tablosu oluşturulur ve “tablodaki değer kadar kaydır” denir. Tablo, alfabedeki her karakter için bir göz içerir. ASCII karakterler düşünülürse 256 göz vardır; alfabe daha küçükse o kadar. Hocanın örneğinde alfabe yalnız A, B, C, D, E, F harflerinden oluşuyordu ve desen yalnız A, B, C karakterlerinden oluşan 3 uzunlukta bir desendi. Desende olmayan D, E, F görülünce her seferinde 3 kaydırılır.

Tablo iki adımda kurulur:

- $i = 0$’dan alfabe boyutu eksi 1’e kadar bütün gözlere m yazılır.
- $j = 0$’dan $m - 2$’ye kadar, desenin j konumundaki karakterin gözüne $m - 1 - j$ yazılır. Buradaki -1, indislerin sıfırdan başlamasından gelir.

Aynı karakter desende birden fazla geçerse, sonraki (daha sağdaki) geçiş öncekinin değerinin üzerine yazar. Böylece “en sağdaki tekrar” kuralı kendiliğinden sağlanır. Desenin son karakteri ($j = m - 1$) döngüye girmez.

5.4 Arama döngüsü

Tablo hazırlandıktan sonra arama tek bir döngüyle yapılır. i, desenin son karakterinin hizasındaki metin indisidir ve $m - 1$’den başlar. $i \leq n - 1$ olduğu sürece:

- Eşleşen karakter sayısı $k = 0$ alınır.
- $k \leq m - 1$ ve desenin $m - 1 - k$ konumu metnin $i - k$ konumuna eşit olduğu sürece k artırılır. Yani “tuttu, tuttu, tuttu”; tutmadığı an çıkılır.
- Döngüden $k = m$ olarak çıkıldıysa bütün karakterler eşleşmiştir; desen $i - m + 1$ indisindedir.
- Değilse i, metindeki T[i] karakterinin tablodaki değeri kadar artırılır.

```c
#include <string.h>

/* Horspool kaydırma tablosu: size = alfabe boyutu (ör. 256). */
void shiftTable(const char *pat, int table[], int size) {
    int m = strlen(pat);
    for (int i = 0; i < size; i++)
        table[i] = m;
    for (int j = 0; j <= m - 2; j++)
        table[(unsigned char)pat[j]] = m - 1 - j;
}

/* Horspool arama: desenin ilk geçtiği indisi döndürür, yoksa -1. */
int horspool(const char *text, const char *pat, const int table[]) {
    int n = strlen(text), m = strlen(pat);
    int i = m - 1;
    while (i <= n - 1) {
        int k = 0;
        while (k <= m - 1 && pat[m - 1 - k] == text[i - k])
            k++;
```

```
        if (k == m)
            return i - m + 1;  /* eşleşme */
        i += table[(unsigned char)text[i]];
    }
    return -1;
}
```

Algoritmanın özeti şöyledir:

- m uzunluğundaki desen için, alfabedeki bütün karakterleri kapsayan kötü eşleşme tablosunu (bir arama tablosu, lookup table) oluştur.
- Deseni metnin başına hizala.
- Son karakterden başlayarak sağdan sola karşılaştır. m eşleşme bulana ya da bir eşleşmeme görene kadar devam et.
- Eşleşmezse, hizadaki metin karakterinin tablodaki değeri kadar kaydır.
- Eşleşme bulana ya da metnin sonuna gelene kadar tekrarla.

Bir ön işleme adımı vardır, ondan sonra arama n'lik bir `while` döngüsüyle tamamlanır. Bu yüzden Horspool oldukça hızlı çalışır.

Soru: Bu algoritma metindeki birden fazla geçişi bulamıyor, değil mi? **Cevap:** Bu, gerçekleştirmeye bağlıdır. Koddaki `return`, kodu kısa tutmak için yazılmıştır. Bulunan indis döndürülmek yerine bir yere kaydedilir ve aramaya devam edilir; sonunda "şu indislerde bulundu" diye hepsi döndürülür. Hoca soruyu ikinci olarak "birden fazla deseni aynı anda arıyor muyuz?" diye de anladı ve onu da cevapladı: Normal algoritmada tek desen aranır. Her desen için ayrı kaydırma tablosu tutan bir gerçekleştirme yapılabilir. Ama çoklu desen araması (multi-pattern search) farklı dinamikleri olan ayrı bir araştırma konusudur. Özellikle biyoinformatikte, gen ağlarında arama için farklı algoritmalar tercih edilir.

5.5 Örnek: BARBER araması

Desen `BARBER` ($m = 6$). Alfabe İngiliz alfabesinin 26 harfi ve boşluk karakteridir. Tablo şöyle kurulur:

j	Karakter	$6 - 1 - j$	Not
0	B	5	
1	A	4	
2	R	3	
3	B	2	B'nin 5 değerinin üzerine yazılır
4	E	1	
5	R	-	Son karakter, döngüye girmez

Sonuç tablo:

Karakter	A	B	E	R	Diğerleri (boşluk dahil)
Kaydırma	4	2	1	3	6

Hocanın sezgisel okuması: `R` görürsem diğer `R`'yi hizalamak için 3 kaydırırım. `E` görürsem `E`'yi hemen yanına oturtmak için 1 kaydırırım; hep `E` görürsek şansımıza 1'er kaydırarak ilerleriz. `B` görürsem 2, `A` görürsem 4 kaydırırım.

Metin `JIM_SAW_ME_IN_A_BARBERSHOP` (boşluklar `_` ile gösterildi). Desen metnin başına hizalanır ve her denemede desenin son karakteri `R` ile hizadaki metin karakteri karşılaştırılır:

Deneme	Hizadaki metin karakteri	Karşılaştırma	Kaydırma
1	A	R ile A tutmadı; A desende var	4
2	E	R ile E tutmadı	1
3	_	R ile boşluk tutmadı; boşluk desende yok	6

Deneme	Hizadaki metin karakteri	Karşılaştırma	Kaydırma
4	B	R ile B tutmadı	2
5	R	R tuttu; E ile A tutmadı	3 (R'nin değeri)
6	R	R, E, B, R, A, B hepsi tuttu	Bulundu

İlk denemedeki 4 kaydırma, metindeki `A` ile desendeki `A`'yı hizalar; o noktada "BARBER" yazıyor olabilir diye kaçırma olmaz. 5. denemede dikkat: Eşleşmeyen karakter `A` olsa da kaydırma `A`'ya ya da `B`'ye göre değil, ilk karşılaştırılan hizadaki karakter `R`'ye göre yapılır. `R`'nin diğer tekrarı kadar, yani 3 kaydırılır. Desen $i - m + 1$ pozisyonunda bulunur; bu metinde 16 indisidir (python ve C ile doğrulandı).

5.6 Örnek: TOOTH tablosu ve araması

Hoca bu kelimeyi bilerek seçti: değerlerin döngüde nasıl güncellendiğini, yani "en sağdaki tekrara gelecek kadar kaydır" kuralının döngüyle nasıl kendiliğinden oluştuğunu göstermek için. Desen `TOOTH`, $m = 5$, indisler 0-4. Bütün alfabe yerine özet bir tablo kullanıldı; diğer bütün karakterler (`*`) için değer 5'tir.

j	Karakter	$5 - 1 - j$	Tablonun o anki hâli
0	T	4	T = 4
1	O	3	O = 3
2	O	2	O = 2 (3'ün üzerine yazıldı)
3	T	1	T = 1 (4'ün üzerine yazıldı)

$j = 2$'de `O` yeniden görülür; daha sağda bir `O` olduğu için daha az kaydırmak gerekir ve 3 silinip 2 yazılır. $j = 3$'te `T` yeniden görülür; artık 4 kaydırılamaz, 1 yazılır. `H` için değer 5 kalır.

> **Ders sonrası not:** Hoca `H`'yi "5 - 1 - 4 = 0, sıfır olamayacağı için 5" diye açıkladı. Döngü $j = m - 2 = 3$'te biter; son karakter `H` ($j = 4$) döngüye hiç girmez. `H`'nin değeri, ilk adımda bütün gözlere yazılan $m = 5$ olarak kalır. Sonuç aynıdır.

Metin `TRUSTHARDTOOTHBRUSHES` (boşluksuz yazıldı):

Deneme	Hizadaki metin karakteri	Karşılaştırma	Kaydırma
1	T	H ile T tutmadı	1 (T'nin değeri)
2	H	H tuttu, T tuttu, O ile S tutmadı	5 (H'nin değeri)
3	O	H ile O tutmadı	2 (O'nun değeri)
4	T	H ile T tutmadı	1 (T'nin değeri)
5	H	H, T, O, O, T hepsi tuttu	Bulundu

2. denemede eşleşmeyen karakter `S` olsa da kaydırma, ilk karşılaştırılan hizadaki `H`'ye göre yapılır. 3. denemedeki 2 kaydırma, desendeki `T`'yi metindeki `D`'nin hizasına getirir. Desen 9 indisinde bulunur (python ve C ile doğrulandı). Bulunamasaydı kaydırmaya devam edilir, sonunda `-1` döndürülürdü.

5.7 Karmaşıklık

En kötü durum: Yine tekrarlı bir durumdur, ama arama sondan yapıldığı için "ters tekrarlı"dır. Metin yalnız `A`'lardan oluşur. Desen de `A`'lardan oluşur ama başında bir `B` vardır: `BAAAAA` (1 `B` ve 5 `A`). Her denemede sondan `A`'lar tutar, en baştaki `B` tutmaz; m karşılaştırma yapılır. Sonra `A`'nın tablo değeri kadar kaydırılır. `A` desende tekrar ettiği için değeri 1'dir. Her seferinde m karşılaştırma ve 1 kaydırma yapılır; bu kaba kuvvetin aynısıdır. En kötü durum $O(mn)$'dir. Desenin bütün karakterlerinin aynı olması gerekmez; örnek basit olsun diye öyle seçildi.

En iyi durum: 5 uzunluğunda bir desen düşünelim. Her denemede ilk karşılaştırma tutmaz ve hizadaki karakter desende yoktur. Her adımda desen boyu kadar, 5 karakter ileri atlanır. Toplam yaklaşık n/m karşılaştırma yapılır. Bu alt doğrusaldır (sublinear): katsayısı $1/m$ olan

n. Büyük O gösteriminde katsayı yazılmadığı için buraya O yazmamak daha doğrudur; en iyi durumda n/m mertebesindedir.

Ortalama: En iyi durum her zaman oluşmaz. Gerçek hayatta Horspool'un karmaşıklığı ortalamada doğrusal, n mertebesinde kalır. Ön işlemedeki m'lik `for` döngüsü de unutulmamalıdır; kabaca $m + n$ denebilir. m çok küçük olduğu için genellikle göz ardı edilir.

Gerçekleştirmesi çok basittir: ön işlemede bir `for` ile tablo, sonra bir `while` ile "eşleştir, eşleşmiyorsa tablodaki değer kadar kaydır". Çok kolay ve oldukça etkili bir alt dizgi arama algoritmasıdır.

6. Boyer-Moore Algoritması

6.1 Horspool ile ilişkisi ve iki sezgi

Horspool, kendisinden önce ortaya çıkmış Boyer-Moore algoritmasının basitleştirilmiş hâlidir. Horspool algoritmayı sadeleştirmiş ve gerçekleştirmesini kolaylaştırmıştır. Boyer-Moore daha karmaşıktır. Bu karmaşıklık şuradan gelir: Horspool tek bir duruma göre tabloya bakıp kaydırıyordu. Boyer-Moore da kötü eşleşme tablosunu kullanmaya devam eder. Ama özellikle tekrarlı desenlerde başka bir kaydırma imkânı olabileceği için bir sezgi daha kullanır: **iyi son ek (good suffix) sezgisi**.

Fikir şudur: Sondan karşılaştırırken birden uzun bir son ek (suffix) eşleşmiş olabilir; örneğin iki ya da üç karakterlik bir eşleşme. Bu son ek desenin başka bir yerinde, hatta en başında da geçiyorsa, deseni o tekrara gelecek ya da o kısmı aşacak kadar kaydırabiliriz. Bunun için ön işlemede bir iyi son ek tablosu daha hesaplanır.

Böylece elimizde iki kaydırma önerisi olur: kötü eşleşme tablosunun söylediği ve iyi son ek tablosunun söylediği. Boyer-Moore hangisi deseni daha ileriye taşıyorsa, yani hangisi daha büyükse onu kullanır. İki tablo da baştan hesaplanıp bir kenara konur; her eşleşmemede ikisinin maksimumu kadar ilerlenir.

> **Ek bilgi (derste anlatılmadı):** Boyer-Moore algoritması büyük olasılıkla 1977'de Robert S. Boyer ve J Strother Moore tarafından yayımlandı. Literatürde "Boyer-Moore-Horspool" adı genellikle Horspool'un 1980'deki sadeleştirilmiş algoritması için kullanılır. Derste "Horspool" diye anlatılan tek tablolu yöntem budur; "Boyer-Moore" diye anlatılan ise iki tablolu yöntemdir.

6.2 Birinci kaydırma: kötü karakter (d_1)

Durumlar şöyledir. k, sondan eşleşen karakter sayısıdır.

- $k = 0$**:** Desenin son karakteri, hizadaki metin karakteri c ile eşleşmez. Boyer-Moore burada Horspool ile aynı davranır: tablodaki değer kadar kaydırır.
- $0 < k < m$**:** Sondan k karakter tutmuş, k indisindeki karşılaştırmada metindeki c karakteri tutmamıştır. Boyer-Moore burada Horspool'dan farklılaşır ve iki değere bakar.

Birinci değer kötü karakter kaydırmasıdır. c desende yoksa desen c'nin bir sağına ($c + 1$'e) taşınır. c tabloda bir değere sahipse kaydırma $t_1(c) - k$ olur: tablodaki değer eksi eşleşen karakter sayısı.

Örnek 1: Desen `BARBER`. `R` tuttu, `E` tuttu, sonra metindeki `S` tutmadı. `S` desende yok; desen `S`'nin sağına taşınır. Bu, $6 - 2 = 4$ karakter kaydırmaktır. Horspool olsaydı hizadaki `R`'nin değeri kadar, yani 3 kaydırırdı. Burada 1 karakter daha fazla kaydırılmış olur; duruma göre bu kazanç m'ye yaklaşacak kadar büyük olabilir.

Örnek 2: Desen `BARBER`. `R` tuttu, `E` tuttu, sonra metindeki `A` ile desendeki `B` tutmadı. `A` desende var. Tablodaki değeri 4, eşleşen karakter sayısı 2: $4 - 2 = 2$ kaydırılır. Bu, metindeki `A`'yı desendeki `A` ile hizalar; bu taraf Horspool gibi çalışır.

Burada bir tehlike vardır: $t_1(c) - k$ sıfır ya da negatif olabilir. Desen geri çekilemez. O zaman kaba kuvvet gibi en az 1 karakter ilerletilir. Birinci kaydırma önerisi şöyledir:

$$d_1 = \max\left(t_1(c) - k,\ 1\right)$$

Ders sonrası not: Hoca Örnek 1'deki 4 karakterlik kaydırmayı anlatırken "good suffix sayesinde" dedi. Bu 4 değeri kötü karakter kaydırması d_1'dir. İki örnekte de iyi son ek kaydırması hesaplanmadı. `BARBER` için $k = 2$ son eki `ER` desende başka yerde geçmez ve desenin hiçbir öneki bu son ekin sonuyla örtüşmez; bu yüzden $d_2 = 6$'dır. Tam Boyer-Moore iki örnekte de $\max(d_1, d_2) = 6$ kaydırır (python ile doğrulandı).

6.3 İkinci kaydırma: iyi son ek (d_2)

Sondan $k > 0$ karakter eşleşmişse, eşleşen bu kısma k uzunluğunda son ek denir. Kötü eşleşme tablosunda olduğu gibi, desendeki bütün son ekler için bir iyi son ek kaydırma tablosu oluşturulur.

Slayttaki örnek desen `ABCBAB`'dir:

- $k = 1$, son ek `B`: desendeki en sağdaki diğer `B`, 2 sola düşer. Bu tablo `B` için "2 kaydır" der; buna d_2 (distance 2) diyelim.
- $k = 2$, son ek `AB`: `AB` desenin başında da geçer. Sondaki `AB`'yi baştaki `AB`'ye taşıyacak kadar, yani 4 karakter birden kaydırabiliriz. $d_2 = 4$.

Son ekin tekrarı varsa iyi son ek bize "tatlı" bir kaydırma verir. Peki tekrarı yoksa ne yapacağız?

Desen `ABCBAB` olsun. Metinden sırayla `B`, `A`, `B` gelip tuttu; ardından gelen `X` karakteri desendeki `C` ile tutmadı. Eşleşen son ek `BAB`'dir ve desende tekrarı yoktur. "Tekrarı yok, deseni boyu kadar kaydırayım" dersek bu yanlış olabilir. Metin örneğin `...BABCBAB...` biçiminde devam ediyorsa (hoca metni `BABCBAB` diye yazdı), deseni boyu kadar taşıdığımızda metinde başlayan `ABCBAB` eşleşmesini kaçırırız. Çünkü `BAB`'nin sonu olan `AB`, desenin başı olan `AB` ile örtüşür.

Demek ki tekrarı olmayan son eklerde kaydırırken bir sınır koymak gerekir. Sınır kolaydır: O ana kadar, daha kısa son ekler için bulunan değer neyse onu koruyun, ondan fazla kaydırmayın. Böylece kaçırma olasılığı ortadan kalkar. `ABCBAB`'de $k = 1$ ve $k = 2$ için (`B` ve `AB`) tekrar eden son ekler vardır. $k = 3, 4, 5$ için son ekler tekrar etmez; bu durumda en uzun tekrar eden kısım kadar kaydırılır.

k	Son ek	d_2	Açıklama
1	B	2	Diğer B'ye hizala
2	AB	4	Baştaki AB'ye hizala
3	BAB	4	Tekrarı yok; AB'yi başa oturt
4	CBAB	4	Tekrarı yok; AB'yi başa oturt
5	BCBAB	4	Tekrarı yok; AB'yi başa oturt

Bu, ikinci arama tablosudur (python ile doğrulandı).

6.4 Karar kuralı

Elimizde iki tablo, iki uzaklık değeri vardır: kötü eşleşme tablosundan gelen d_1 ve iyi son ek tablosundan gelen d_2. Karar şöyledir:

- $k = 0$ ise, yani sondan hiç eşleşme yoksa, doğrudan d_1 kullanılır. Bu Horspool'un çalışma biçimidir.
- $k > 0$ ise $d = \max(d_1, d_2)$ kadar kaydırılır.

$$d = \begin{cases} d_1 & k = 0 \\ \max(d_1, d_2) & k > 0 \end{cases}$$

6.5 Örnek: BAOBAB araması

Metin `BESS_KNEW_ABOUT_BAOBABS`, desen `BAOBAB` ($m = 6$).

Kötü eşleşme tablosu: Desende `B`, `A` ve `O` vardır. `A` görülünce 1 kaydırılır (`A`'yı hizala). `B` görülünce 2 kaydırılır (en sağdaki diğer `B`'yi hizala). `O` görülünce 3 kaydırılır. Diğer her şey için 6. Bu taraf Horspool'dur.

Karakter	A	B	O	Diğerleri
t_1	1	2	3	6

İyi son ek tablosu: $k = 1$ için tek karakter (`B`) olduğundan değer kötü eşleşme tablosundakiyle aynı çıkar: 2. Tek karakterden fazla eşleşmede iyileştirme gelebilir. `AB` desende başka yerde tekrar etmez. Ama başta `B` olduğu için, sondaki `B`'yi baştaki `B`'ye taşıyacak kadar, 5 karakter kaydırabiliriz. `BAB`, `OBAB`, `AOBAB` için de tekrar yoktur; yine baştaki `B` hizalanır ve 5 kaydırılır.

k	Son ek	d_2
1	B	2
2	AB	5
3	BAB	5
4	OBAB	5
5	AOBAB	5

Hoca tabloyu yorumlarken şunu vurguladı: `AB` eşleştikten sonra bir eşleşmeme olursa, Horspool sonraki `B`'ye bakıp yalnız 2 kaydırırdı. Şimdi "5 kaydırabilirsin" diye yeni bir öneri vardır. 2 mi 5 mi? Tabii ki 5. Fazladan 3 karakter ileri atlanır.

Arama:

Deneme	Karşılaştırma	k	d_1	d_2	Kaydırma
1	B ile K tutmadı	0	$t_1(K) = 6$	-	6
2	B tuttu, A tuttu; B ile boşluk tutmadı	2	$6 - 2 = 4$	5	5
3	B tuttu; A ile boşluk tutmadı	1	$6 - 1 = 5$	2	5
4	B, A, B, O, A, B hepsi tuttu	-	-	-	Bulundu

1. denemede $k = 0$'dır. Hoca iyi son ek tablosuna bakmaya gerek olmadığını söyledi: `K` desende yoktur ve eşleşen bir son ek oluşmaz. Boyer-Moore Horspool gibi davranır ve 6 kaydırır.
2. denemede desen `NEW_AB` kısmının hizasındadır; tabii algoritma bunu henüz bilmez. `B` ve `A` tuttu, üçüncü karşılaştırmada boşluk tutmadı. $d_1 = 6 - 2 = 4$, $d_2 = 5$'tir. 5 daha kârlıdır; sondaki `B`, metindeki `B`'nin hizasına gelir.
3. denemede `B` tuttu. Boşluk (slaytta alt çizgi) ile `A` tutmadı. $d_1 = 6 - 1 = 5$, $d_2 = 2$'dir. Bu kez durum tersinedir; kötü eşleşme tablosu 5, iyi son ek 2 der. Tabii ki 5 seçilir. Ardından bütün karakterler tutar ve desen bulunur. Desen 16 indisindedir (python ile doğrulandı). Metnin sonuna kadar gitme kontrolü de kolayca eklenebilir.

6.6 Horspool ile Boyer-Moore'u ayırt etmek ve karmaşıklık

> **Hocanın vurgusu:** İnternette Horspool ve Boyer-Moore birbirine çok karışık şekilde verilir; insanlar Horspool diye Boyer-Moore'u verebilir. Gerçek ayrım şudur: Ön işlemede hem kötü eşleşme tablosu hem iyi son ek tablosu oluşturulup, karar anında "4 mü kaydırayım 5 mi" diye ikisinin büyüğü seçiliyorsa o kod Boyer-Moore'dur. Yalnız kötü eşleşme tablosu oluşturulup, başka bir tablo olmadan yalnız ondaki değere göre kaydırılıyorsa o kod Horspool'dur.

Horspool çok daha kolay gerçekleştirilir. Boyer-Moore'da iyi son ek tablosu için biraz daha iş yapmak, karar verirken de iki uzaklığı birden alıp kârlı olanı seçmek gerekir.

Boyer-Moore'un en kötü durumu, her seferinde Horspool gibi davranmasıdır. En kötü durumda yine mn'ye çıkabilir. Ama genellikle Horspool'dan daha fazla kaydırdığı için toplam iş biraz daha azdır; ortalamada doğrusal zamanda kalır. En kötü durumun oluşması için özel seçilmiş bir desen ve metin gerekir. Gerçek hayattaki normal metinlerde Boyer-Moore, Horspool'dan da daha etkili biçimde doğrusal zamanda arar.

7. Rabin-Karp Algoritması

7.1 Fikir: imzayı aramak

Rabin-Karp, çalışma mantığı bakımından önceki algoritmalardan farklıdır. Kaba kuvvette de Horspool ve Boyer-Moore'da da deseni metin üzerinde karakter karakter karşılaştırarak ilerliyorduk. Buradaki fikir bunun tersidir: Metni doğrudan aramayalım. Desenin bir imzasını, özetini çıkaralım ve bu imzayı arayalım; yani sayısal bir arama yapalım.

İmza, desenin bir özet (hash) fonksiyonundan geçirilmiş hâlidir. Fonksiyonun çıktısı desenin imzasıdır. Metinde bu imza aranır. İmza bulunamazsa desen metinde kesinlikle yoktur.

Özet fonksiyonu "fonksiyon" diye anılsa da geri dönüşü olmayan, tek yönlü bir dönüşümdür. Desen belirli matematik işlemlerinden geçirilir ve bir çıktı, yani imza elde edilir. Girdi değiştiğinde farklı çıktı üretecek şekilde çalışması istenir.

Burada bir tehlike vardır. Özet fonksiyonu yeterince güçlü, yani yeterince farklı değer üretebilecek biçimde seçilmezse çıktılar çakışabilir (collision): farklı desenler aynı imzayı üretebilir. İmza tuttuğunda "buldum" derseniz, aradığınız deseni değil, aynı imzayı veren başka bir dizgiyi bulmuş olabilirsiniz. Çözümü kolaydır: İmza eşleşmesi yakalandığında o indisten sonraki m karakter alınır ve desenle birebir karşılaştırılır. Yalnız m karakterlik bir karşılaştırmayla gerçekten tutup tutmadığı garanti edilir.

Arama yapıldığı için yalnız desenin değil, metindeki her pozisyonun (m uzunluğundaki her pencerenin) özetinin de hesaplanıp desenin özetiyle karşılaştırılması gerekir.

Karmaşıklık:

- Ön işlemede desenin özetini çıkarmak $O(m)$'dir.
- En kötü durumda n kez imza tutar ve her seferinde şans eseri başka bir dizgi o özeti üretmiş olur. Her seferinde m karakter kontrol edilir; mn'ye gidilir. Bu çok çok düşük bir olasılıktır; özet fonksiyonu düzgün seçilirse böyle bir durum olmaz.
- Genellikle $m + n$ mertebesinde kalır. m toplamda önemli değildir; asıl ağırlık n'dedir. Arama doğrusal zamanda yapılır.

7.2 Örnek: π'nin basamaklarında 59265

Hoca, özet fonksiyonunu kolay hesaplamak için deseni ve metni sayısal seçti. Normalde metin de olabilir; karakterlerin sayısal karşılıkları özetten geçirilir. Burada çok basit bir özet fonksiyonu kullanıldı: sayının bir asal sayıya göre modunu almak.

Metin π sayısının basamaklarıdır: `3141592653589793`. Desen `59265` ($m = 5$). Asal sayı $q = 97$.

Önce ön işleme: desenin özeti.

$$59265 \bmod 97 = 95$$

Bu, desenin imzasıdır. Sonra metnin ilk 5 karakteri okunur, özeti hesaplanır ve 95 ile karşılaştırılır. Tutmazsa bir kaydırılır:

Pencere	Özet (mod97)	95 ile karşılaştırma
31415	84	Tutmadı; bu pencere desene eşit olamaz
14159	94	Tutmadı
41592	76	Tutmadı
15926	18	Tutmadı
59265	95	Tuttu

Son pencerede aynı özete sahip bir değer bulundu. Bu aşamada karakterler teker teker kontrol edilir: 5-5, 9-9, 2-2, 6-6, 5-5. m karakterin hepsi tuttu; desenin gerçekten burada olduğu, birebir eşit olduğu da test edildi. Bütün değerler python ile doğrulandı.

> **Ders sonrası not:** Hoca mod 97'nin "1 ile 97 arasında ya da 0 ile 96 arasında" bir değer döndüreceğini söyledi. x mod 97 her zaman 0 ile 96 arasındadır.

Ders sonrası not: Hoca "alfabe 96 karakterden azsa herhalde çakışma yaşamam" dedi. Çakışma alfabe boyutuna değil, olası pencere sayısına bağlıdır. 5 basamaklı 10^5 farklı pencere yalnız 97 farklı özete düşer; çakışma kaçınılmazdır. Bu örnekte bile desenden sonraki pencere 92653'ün özeti 18'dir; daha önce görülen 15926 ile aynıdır (python ile doğrulandı).

7.3 Kayan özet (rolling hash)

Soru: Her adımda özet hesaplıyoruz; n tane özet işlemi maliyet getirmiyor mu? **Cevap:** Getiriyor. Ama işin güzelliği şuradadır: Pencere birer kaydırıldığı için, bir sonraki adımın özeti önceki adımın özetinden hesaplanabilir. Yalnız ilk özet bir maliyettir; sonrası çok basit işlemlerdir.

İlk sayı 31415'tir. Bundan kaçış yoktur; $31415 \bmod 97 = 84$ hesaplanır. Bir sonraki pencerede baştaki 3 düşer, sona 9 gelir:

$$14159 = (31415 - 3 \cdot 10000) \cdot 10 + 9$$

Önceki değerden düşen basamağın katkısı (30000) çıkarılır, kalan 1415 bir basamak sola kaydırılır (10 ile çarpılır, sona bir sıfır gelir) ve yeni gelen basamak 9 eklenir. Aynı işlem mod altında da yapılabilir. Önceki adımın modu (84) zaten bilinir; yeniden hesaplamaya gerek yoktur.

$30000 \bmod 97$ için $10000 \bmod 97 = 9$ önceden hesaplanır. O zaman $30000 \bmod 97$ yerine $3 \cdot 9 = 27$ kullanılır. Basamak 0'dan 9'a kadar on olasılık olduğu için bu çarpımlar önceden bir tabloya yazılıp oradan maliyetsiz okunabilir.

$$h = \big((84 - 3 \cdot 9) \cdot 10 + 9\big) \bmod 97 = (57 \cdot 10 + 9) \bmod 97 = 579 \bmod 97 = 94$$

Maliyet bir çıkarma, bir çarpma, 10 ile bir çarpma ve bir toplamadır; ardından küçük bir sayıyla bir mod işlemi yapılır. Hoca 10 ile çarpmanın daha etkili kodlanabileceğini de belirtti. Genel biçim şöyledir (t_i, pencereden düşen basamak):

$$h_{i+1} = \Big((h_i - t_i \cdot (10^{m-1} \bmod q)) \cdot 10 + t_{i+m}\Big) \bmod q$$

Sonraki adımlar hep bin'den küçük sayılarla yapılır:

Önceki özet	Düşen	Gelen	İşlem	Yeni özet
84	3	9	$(84 - 27) \cdot 10 + 9 = 579$	94
94	1	2	$(94 - 9) \cdot 10 + 2 = 852$	76
76	4	6	$(76 - 36) \cdot 10 + 6 = 406$	18
18	1	5	$(18 - 9) \cdot 10 + 5 = 95$	95

Son adımda 95 tutar ama bu yeterli değildir; karakterler teker teker kontrol edilir ve hepsi tutar (ara adımlar eklendi; değerler python ile doğrulandı).

Ders sonrası not: Hoca formülü sözlü olarak "84 eksi 3 çarpı 9 çarpı 10, artı 9" diye okudu. Doğru işlem sırası $\big((84 - 3 \cdot 9) \cdot 10 + 9\big) \bmod 97$'dir: önce çıkarma, sonra 10 ile çarpma. Bu örnekte parantez içi hep pozitif çıkıyor. Genel durumda $h_i - t_i \cdot (10^{m-1} \bmod q)$ negatif olabilir; o zaman sonuca q eklenerek pozitife çevrilir.

7.4 Asal sayının seçimi ve kullanım alanı

Özetler bir kez hesaplandığı için, aynı anda birden fazla değer aranacaksa Rabin-Karp ile çok daha hızlı karşılaştırma yapılabilir. Özet fonksiyonu çakışmaları engelleyecek kadar güçlü ve bir sonraki adımı öncekinden hesaplanabilecek biçimde seçilirse, Rabin-Karp ile de doğrusal zamanda çok etkili arama yapılır.

Hocanın vurgusu: Buradaki anahtar nokta seçilen asal sayıdır. Yeterince büyük seçilmelidir. Çok küçük seçilirse (örneğin 7) çakışma çok olur. Aynı özet değeri birçok kez görülür ve her birinde fazladan m adımlık bir kontrol yapmak gerekir. Bunu en aza indirmek için asal sayı büyük seçilmelidir.

Ders sonrası not: Hoca algoritmanın adını "Robin-Karp" diye söyledi. Doğrusu Rabin-Karp'tır (Michael Rabin ve Richard Karp).

8. Algoritmaların Karşılaştırılması ve Seçim

Hoca son olarak slayttaki karşılaştırma tablosunu ara çizgilerini çizerek anlattı. Tabloda Horspool ayrıca yer almaz. Boyer-Moore yer alır ve Horspool ona neredeyse eşittir. Hocaya göre harika bir algoritmadan bir özelliği çıkarıp yeni bir algoritma olarak sunmak garip görünebilir; ama sonuçta Horspool'un gerçekleştirmesi gerçekten çok daha basittir. Adlandırma da farklıdır: biri "mismatched character", öbürü "bad match" der.

Algoritma	Garanti (en kötü)	Tipik	Geri dönüş (backup)	Doğruluk	Ek alan
Kaba kuvvet	mn	$1.1n$	Gerekli	Garantili	1
KMP (tam DFA)	$2n$	$1.1n$	Gerekmez	Garantili	mR
KMP (mismatch geçişli)	$3n$	$1.1n$	Gerekmez	Garantili	-
Boyer-Moore (yalnız kötü karakter)	mn	n/m	Gerekli	Garantili	R
Rabin-Karp (Monte Carlo)	$7n$	-	Gerekmez	Özet fonksiyonuna bağlı	1
Rabin-Karp (Las Vegas)	-	-	Gerekli	-	1

R alfabe boyutudur. Tabloyu satır satır hocanın yorumuyla okuyalım:

- **Kaba kuvvet:** En kötü durumda mn'yi garanti eder. Gerçek hayatta tipik olarak $1.1n$ mertebesinde çalışır. Mutlaka geri dönüş alanı gerekir. Garantili olarak bulur. Tabloda ek alan 1 yazar; hocaya göre buraya m yazmak daha doğrudur. Geri dönüş yoksa 1, varsa m denebilir.
- **KMP:** Derste anlatılmadı; DFA (deterministik sonlu otomat) temelli bir algoritmadır. DFA ile çalıştığında $2n$'yi, mismatch geçişleriyle çalışan sürümde $3n$'yi garanti eder. Tipik olarak $1.1n$'dir. Geri dönüş alanı istemez ve doğrudur. Ama ek alan ihtiyacı fazladır: $m \cdot R$.
- **Boyer-Moore (yalnız kötü karakter, yani Horspool sürümü):** En kötü durumda mn, tipik olarak n/m mertebesindedir. Bu, diğerlerinden çok daha iyi bir performanstır. Bu yüzden Horspool birçok arama kutusunda (search box) standart olarak kullanılır. Zayıf karnı geri dönüş ihtiyacıdır: son karakterden ilk karaktere doğru gidildiği için m'lik bir geri dönüş alanı gerekir. Ek alan, karakter sayısı kadar olan tablodan gelir.
- **Rabin-Karp:** Doğruluk ve garanti özet fonksiyonuna bağlıdır. Monte Carlo ve Las Vegas sürümleri vardır; derste düz mod alma yöntemi anlatıldı. Monte Carlo sürümü $7n$'yi garanti eder; hoca ayrıntısına girmedi. Geri dönüş gerekmez, çünkü "kaydır, tuttu mu" diye özet üzerinden ilerler. Las Vegas sürümü anlatılmadı; özetin hesaplanması farklıdır ve bu yöntemde geri dönüş alanı gerekir. Ek alan sabittir: bir sonraki adım önceki özetten hesaplandığı için yalnız önceki değeri tutmak yeterlidir.

  Ek bilgi (derste anlatılmadı): Rabin-Karp'ın Monte Carlo sürümü, özetler eşleşince karakterleri kontrol etmeden "bulundu" der. Bu yüzden her zaman hızlıdır ama çok küçük bir olasılıkla yanlış sonuç verebilir. Las Vegas sürümü özet eşleşince karakterleri tek tek kontrol eder; sonucu her zaman doğrudur ama süresi olasılıksaldır. Derste π örneğinde yapılan "özet tutunca m karakteri kontrol et" adımı Las Vegas yaklaşımına karşılık gelir. Karakter kontrolü için metnin geriye doğru okunması

gerektiğinden Las Vegas sürümünün tabloda geri dönüş istemesi büyük olasılıkla bundandır.

Görüldüğü gibi algoritmaların biri ekstra yer ister, biri geri dönüş ister, biri daha etkili çalışır. Aralarında bir ödünleşim (trade-off) yapmak gerekir. Hocanın önerileri:

- Bazen yalnız kaba kuvvet (naif) arama kullanılır.
- Arama kutularında, metin söz konusu olduğunda en çok Boyer-Moore (ve Horspool) kullanılır.
- Akan verilerde KMP kullanılabilir. KMP aramaya baştan başlar ve ön işlemede "kaybetmeden en fazla kaç karakter kaydırabilirim" tablosunu kurar; soldan sağa çalıştığı için geri dönüş istemez. Ağda veri tutmadan, geri dönüş kullanmadan bir şey yapmak gerekiyorsa en iyi yöntemdir.
- Çoklu arama gerekiyorsa Rabin-Karp ve özet tabanlı türevleri kullanılır.

Hoca, öğrencilerin Boyer-Moore, Horspool ve Rabin-Karp'ı kendileri kodlayıp kurcalamasını önerdi. Kodları Wikipedia başta olmak üzere birçok kaynakta vardır. Karşılaştırırken Boyer-Moore ile Horspool'un karışabileceğini, dersteki anlatıma göre bakmanın daha iyi olacağını hatırlattı. Anlatılanların hiçbiri gizli bir bilgi değildir; önemli olan, yöntemlerin getirdiği bakış açısıdır: Kaba kuvvet aramayı bir ön işleme adımıyla nasıl iyileştirir ve daha fazla adımı nasıl atlarız?

Örnek Sorular

1. (derste sözlü, 00:19:40) Kaba kuvvet aramada bir karşılaştırma tutmadığında desen neden yalnız bir karakter kaydırılır? Daha çok kaydıramaz mıyız?

> **Çözüm:** Kaydıramayız. Desen bir sonraki karakterden ya da ondan sonrakinden itibaren başlıyor olabilir; fazla kaydırırsak bu eşleşmeyi kaçırırız. Kaba kuvvette her konum tek tek denenmelidir.

2. (derste sözlü, 00:35:07) Horspool sondan karşılaştırdığına göre, akan veride ek bir alana ihtiyaç duyar mı?

> **Çözüm:** Evet. Desenin son karakterinin hizasındaki karakter en son gelen veridir. Önceki karakterlerle de karşılaştırma yapıldığı için yaklaşık m karakterlik bir geri dönüş (backup) alanı gerekir.

3. (derste sözlü, 00:41:57) `REORDER` deseninde `R` tuttuktan sonra metindeki `A` ile `E` tutmadı. `A` desende olmadığı hâlde Horspool neden deseni daha fazla kaydırmaz?

> **Çözüm:** Horspool kaydırmayı eşleşmeyen karaktere göre değil, desenin son karakteri hizasındaki metin karakterine (`R`) göre yapar. Tablo tek kuralla kurulur ve `R`'nin değeri $7 - 1 - 3 = 3$'tür. Desen 3 kaydırılır. Bu, Horspool'un olması gerekenden az kaydırdığı zayıf noktasıdır. (Derste 4 söylendi; doğrusu 3'tür.)

4. (derste sözlü, 00:49:43) Horspool metindeki birden fazla geçişi bulabilir mi?

> **Çözüm:** Gerçekleştirmeye bağlıdır. Eşleşme bulununca `return` etmek yerine indis kaydedilir ve aramaya devam edilir. Aynı anda birden fazla farklı desen aramak ise ayrı bir konudur (çoklu desen araması); normal algoritma tek desen arar.

5. (derste sözlü, 01:24:13) Boyer-Moore'da $t_1(c) - k$ sıfırın altına düşerse desen geri mi çekilir?

> **Çözüm:** Hayır, desen geri çekilemez. En az 1 kaydırılır: $d_1 = \max(t_1(c) - k, 1)$.

6. (derste sözlü, 01:34:11) `BAOBAB` aranırken `AB` eşleşti, bir sonraki karakter tutmadı. Kötü karakter tarafı 2, iyi son ek tablosu 5 kaydırmayı öneriyor. Hangisi seçilir?

> **Çözüm:** $k > 0$ olduğu için $\max(d_1, d_2)$ seçilir; 5 kaydırılır. Derste yapılan aramanın 2. denemesinde de $d_1 = 6 - 2 = 4$, $d_2 = 5$ idi ve 5 seçildi.

7. (derste sözlü, 01:50:10) Mod 97 kullanılan Rabin-Karp örneğinde, alfabe 96 karakterden küçükse çakışma yaşanmaz mı?

Çözüm (derste verilmedi): Yaşanır. Çakışma alfabe boyutuna değil, olası pencere sayısına bağlıdır. 5 basamaklı 10^5 pencere yalnız 97 özete düşer. Aynı metinde `15926` ve `92653` pencerelerinin ikisinin de özeti 18'dir (python ile doğrulandı). Bu yüzden özet tuttuğunda m karakter yine kontrol edilir ve pratikte büyük bir asal seçilir.

8. (derste sözlü, 01:53:43) Rabin-Karp'ta her pencere için özet hesaplamak n tane pahalı işlem getirmez mi?

Çözüm: Kayan özet ile getirmez. $h_{i+1} = \big((h_i - t_i \cdot (10^{m-1} \bmod q)) \cdot 10 + t_{i+m}\big) \bmod q$. Örnekte $\big((84 - 3 \cdot 9) \cdot 10 + 9\big) \bmod 97 = 94$. Her adım küçük sayılarla bir çıkarma, iki çarpma, bir toplama ve bir mod işlemidir.

Dersin Toparlaması

Bu derste n uzunluğundaki bir metinde m uzunluğundaki bir deseni bulma problemi ele alındı. Kaba kuvvet arama her konumu dener; eşleşme bozulunca yalnız bir kaydırır, en kötü durumda mn karşılaştırma yapar ve akan veride m karakterlik geri dönüş alanı ister. Horspool, deseni sağdan sola karşılaştırır ve hizadaki metin karakterinin kötü eşleşme tablosundaki değeri kadar kaydırır. Gerçekleştirmesi çok basittir, en iyi durumda n/m'ye iner ve bu yüzden arama kutularında standarttır. Boyer-Moore buna iyi son ek tablosunu ekler. Sondan eşleşme varsa $\max(d_1, d_2)$ kadar kaydırır; iki tablolu kod Boyer-Moore, tek tablolu kod Horspool'dur. Rabin-Karp karakterleri değil, mod alma ile çıkarılan özetleri karşılaştırır. Kayan özet sayesinde her adım sabit maliyetlidir. Çakışma olabileceği için özet tutunca karakterler tek tek kontrol edilir ve asal sayı büyük seçilir.

Hoca kapanışta, hangi algoritmanın seçileceğinin bir ödünleşim olduğunu vurguladı: metin kutularında Boyer-Moore ve Horspool, akan veride geri dönüş istemeyen KMP, çoklu desen aramasında Rabin-Karp ve özet tabanlı türevleri öne çıkar. KMP'yi bu yıl anlatamadığını, istekli öğrencilerin dersteki bakış açısıyla bu algoritmaları kendilerinin kodlayıp denemesini önerdi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.